



Travail pratique #1

IFT-4022/IFT-7022

Modèles de langue et classification de textes

Équipe 18 :

Hedi BOUKAMCHA - 537401873

Antonin CREIGNOU - 537448254

Valentin JONCHERAY - 537466177

Présenté à :

Luc Lamontagne

Automne 2026

Université Laval

Québec, Canada

1 Tâche 1 – Modèles de langue N-grammes : on connaît la chanson

1.1 Sous-tâche 1.1 – Choisir le titre connu dans une liste de candidats (estimation)

1.1.1 Objectif

Pour chaque test du fichier `t1_estimation_test.json`, on dispose de 3 candidats, dont un seul est un vrai titre de chanson. Il faut construire des modèles de langue N-grammes de mots ($N = 1, 2, 3$) avec lissage de Laplace, entraînés sur les 572 titres de `t1_titres_chansons.txt`, puis utiliser chaque modèle pour retenir le candidat qui lui paraît le plus vraisemblable. On mesure ensuite la précision de chaque modèle sur les 20 tests.

1.1.2 Démarche

Construction des modèles : Les titres sont mis en minuscules, tokenisés avec `word_tokenize`, puis complétés par des symboles de début et de fin de titre (`<BOS>`, `<EOS>`). Les fonctions `build_vocabulary` et `get_ngrams` sont reprises de l'exemple NLTK du cours. La fonction `train_models` construit les trois modèles Laplace de NLTK en un seul appel et les range dans le dictionnaire `models`. Le vocabulaire compte $V = 441$ mots, symboles spéciaux compris.

Choix du candidat : Pour chaque candidat, `select_candidate` calcule sa **perplexité** avec le modèle choisi et retient celui dont la perplexité est la plus basse. Nous avons préféré la perplexité à la probabilité brute du titre, car les candidats n'ont pas tous la même longueur (par exemple « i gotta feeling » contre « i gotta a feeling to rock you ») : la probabilité brute pénaliserait systématiquement les plus longs.

Évaluation : Pour chaque modèle, on compte le nombre de tests où le candidat retenu est le titre attendu (champ `Song`).

1.1.3 Résultats

Modèle	Bonnes réponses	Précision
Unigramme ($N = 1$)	6 / 20	30 %
Bigramme ($N = 2$)	10 / 20	50 %
Trigramme ($N = 3$)	13 / 20	65 %

TABLE 1 – Précision des trois modèles sur les 20 tests d'estimation.

La précision augmente avec l'ordre du modèle. L'unigramme fait à peine mieux que le hasard (33 % avec 3 candidats). La Table 2 détaille, pour chaque test, si chaque modèle retrouve le bon titre.

Titre attendu	Unigramme	Bigramme	Trigramme
i want to hold your hand	×	✓	✓
let it be	×	×	×
i want to know what love is	×	×	×
i will survive	×	×	×
i will wait	✓	✓	✓
i gotta feeling	×	×	×
i want to break free	×	✓	✓
we will rock you	×	×	×
i kissed a girl	✓	✓	✓
i believe i can fly	×	×	✓
i am a man	✓	✓	✓
you are so beautiful	✓	×	×
let the good times roll	×	✓	✓
let's have a party	×	✓	✓
we are family	✓	✓	✓
we didn't start the fire	×	×	✓
love shack	×	×	×
love the way you lie	✓	✓	✓
my heart will go on	×	×	✓
the sound of silence	×	✓	✓
Total	6 / 20	10 / 20	13 / 20

TABLE 2 – Réussites (✓) et erreurs (×) des trois modèles sur chaque test d'estimation.

Six tests mettent en échec les trois modèles, tandis que d'autres ne sont résolus qu'à partir du bigramme (« i want to break free », « the sound of silence ») ou du trigramme (« i believe i can fly », « my heart will go on »). Le test « you are so beautiful » est le seul que l'unigramme réussit et que les deux autres modèles ratent.

1.1.4 Analyse

Pourquoi la précision augmente avec N : L'unigramme ne tient compte d'aucun contexte et favorise les candidats composés de mots fréquents : il choisit par exemple « love you » plutôt que « love shack ». Le bigramme et le trigramme tiennent compte d'un ou deux mots précédents et captent mieux les enchaînements propres aux vrais titres.

Étude d'un cas difficile : « let it be » contre « let me be » Les trois modèles se trompent sur ce test, alors que « let it be » figure tel quel dans le corpus d'entraînement (Table 3).

Candidat	Unigramme	Bigramme	Trigramme
let it be (titre attendu)	119,6	131,6	143,6
let me be	72,0	90,3	117,8
let it roll	217,3	155,6	189,4

TABLE 3 – Perplexité de chaque candidat pour le test « let it be » (en gras : candidat retenu).

Deux causes se combinent :

- **La fréquence des débuts de titres :** le corpus contient 10 titres commençant par « let me », contre seulement 2 par « let it ». Les modèles jugent donc « let me » beaucoup plus probable.

- **Le lissage de Laplace** : pour le troisième mot, le trigramme a observé « be » 1 fois sur 2 après « let it », et 2 fois sur 10 après « let me ». Sans lissage, « let it be » serait nettement favorisé (50 % contre 20 %). Mais avec un vocabulaire de 441 mots, le lissage écrase ces petits comptes (0,45 % contre 0,67 %) et inverse le classement en faveur du contexte le plus fréquent.

Autres difficultés observées :

- **Certains distracteurs sont eux-mêmes de vrais titres** : « we are so beautiful » figure aussi dans le corpus, ce qui rend le test « you are so beautiful » ambigu (perplexités très proches avec le trigramme : 107,8 contre 104,3).
- **Les débuts de titres fréquents l'emportent** : « i will rock you » est préféré à « we will rock you », car « i » est le premier mot le plus fréquent du corpus.

Limites : Le lissage de Laplace, imposé par l'énoncé, est trop fort pour un corpus aussi petit (572 titres) : il donne trop de poids aux suites jamais observées et efface des différences réelles entre les comptes. Un lissage plus fin (additif avec une constante inférieure à 1, ou Kneser-Ney) ou un corpus plus grand devraient améliorer les résultats.

1.2 Sous-tâche 1.2 – Compléter les titres de chansons (génération de texte)

1.2.1 Objectif

Il s'agit de compléter les titres du jeu de test en générant les mots masqués avec les modèles de la sous-tâche 1.1, puis de mesurer la proportion de mots générés qui correspondent au titre connu.

1.2.2 Démarche

Génération : La fonction `complete_title` génère les mots un par un avec `model.generate` : chaque mot généré est ajouté au contexte du suivant. Trois choix ont été nécessaires :

- le titre incomplet reçoit les mêmes symboles de début qu'à l'entraînement, pour que le modèle reconnaisse un début de titre ;
- les symboles spéciaux (<BOS>, <EOS>, <UNK>) sont exclus, pour que seuls de vrais mots complètent le titre ;
- quand le contexte n'a jamais été observé, on le raccourcit jusqu'à obtenir un contexte connu, et on repart d'un début de titre si aucun ne l'est.

La graine aléatoire est fixée (`random_seed=46`) pour rendre les résultats reproductibles.

Évaluation : Sur les 40 titres du test (66 mots masqués au total), l'exactitude est la proportion de mots générés identiques, à la même position, aux mots du titre connu.

1.2.3 Résultats

Modèle	Exactitude
Unigramme ($N = 1$)	0,0 %
Bigramme ($N = 2$)	22,7 %
Trigramme ($N = 3$)	63,6 %

TABLE 4 – Exactitude de la génération des mots masqués sur les données de test.

Pour comprendre ces scores, nous avons comparé la génération des trois modèles sur trois exemples. Les deux premiers sont des titres du corpus ; le troisième est un contexte inventé, jamais observé.

Titre incomplet	Unigramme	Bigramme	Trigramme
<i>i want to *** *** (break free)</i>	want know	tomorrow let	want me
<i>i believe i *** *** (can fly)</i>	want know	will go	can fly
<i>mot inconnu *** ***</i>	want know	we belong	we let

TABLE 5 – Mots générés par chaque modèle. En vert : complétion correcte ; en rouge : complétion incorrecte ; en gris : contexte inconnu.

1.2.4 Analyse

Unigramme (0 %) : Le modèle ignore complètement le début du titre et tire toujours des mots fréquents du corpus. Il produit donc la même suite (*want know*) pour les trois titres et ne retrouve aucun mot.

Bigramme (22,7 %) : Avec un seul mot de contexte, les suites possibles sont trop nombreuses. Par exemple, « i » est suivi de 14 mots différents dans le corpus : le modèle produit « i will go », un enchaînement plausible localement mais sans lien avec le titre recherché.

Trigramme (63,6 %) : Deux mots de contexte suffisent souvent à identifier un titre du corpus. Après « believe i », le seul mot observé est « can », puis « fly » : le modèle retrouve le titre avec certitude. Il échoue en revanche quand le contexte est partagé par plusieurs titres. « want to » est suivi de 7 mots différents, chacun observé une seule fois : le tirage donne « want », puis « me », ce qui produit « i want to want me », une suite qui combine des fragments de deux titres différents.

Contexte inconnu : Pour « mot inconnu », aucun contexte n'est connu : le modèle repart d'un début de titre à chaque mot. « we let » n'est donc pas une suite apprise, mais deux débuts de titres tirés indépendamment.

Limites :

- La génération est un **tirage aléatoire** : avec une autre graine, les scores changeraient, surtout pour les contextes ambigus comme « want to ».
- Tous les titres du test figurent dans le corpus d'entraînement : ces scores mesurent surtout la capacité à **retrouver des titres connus**. Sur des titres inédits, l'avantage du trigramme serait bien plus faible, car ses contextes de deux mots seraient rarement observés.

2 Tâche 2 – Classification de textes : les classes d'incidents

Cette tâche consiste à classer de courtes descriptions d'incidents de chantiers de construction parmi 9 classes numérotées de 0 à 8, dont le sens est inconnu, puis à interpréter ces classes.

2.1 Démarche

Les textes sont représentés en sac de mots avec un **CountVectorizer** (paramètres par défaut), appris sur l'entraînement seulement. Nous comparons le Naïf Bayésien (**MultinomialNB**)

et la Régression Logistique (`LogisticRegression`) de Scikit-learn, entraînés par la fonction `train_and_test_classifier`.

Comme les classes sont très déséquilibrées et qu'un seul découpage peut favoriser un modèle par hasard, nous complétons l'exactitude sur le test par trois mesures : le F1 par classe et le F1 macro, une validation croisée à 5 plis sur l'entraînement, et un test de la régularisation `C` de la régression logistique.

2.2 Résultats

	Naïf Bayésien	Régression Logistique
Exactitude (entraînement)	85,7 %	100,0 %
Exactitude (test)	67,6 %	69,3 %
Validation croisée à 5 plis	68,5 % $\pm$ 1,3	71,4 % $\pm$ 2,3
F1 macro (test)	0,439	0,541

TABLE 6 – Comparaison des deux classificateurs.

La régression logistique est meilleure sur toutes les mesures. L'écart d'exactitude sur le test est faible (1,7 point), mais la validation croisée confirme qu'il ne dépend pas d'un découpage particulier. L'écart est surtout marqué sur le F1 macro (0,54 contre 0,44), car la régression logistique traite bien mieux les classes rares (Table 7).

Classe	Incidents (test)	F1 – NB	F1 – LR
0	103	0,591	0,597
1	7	0,000	0,444
2	58	0,880	0,883
3	15	0,235	0,500
4	2	0,000	0,000
5	191	0,817	0,851
6	29	0,537	0,652
7	66	0,459	0,459
8	60	0,435	0,482

TABLE 7 – F1 par classe des deux classificateurs sur les données de test.

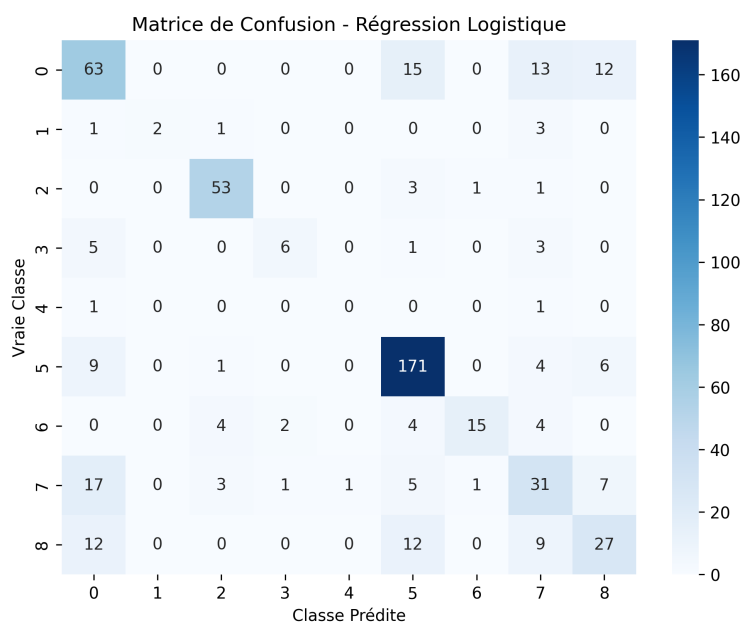


FIGURE 1 – Matrice de confusion de la Régression Logistique sur les données de test (531 incidents).

Analyse de la matrice de confusion :

- **Les classes au vocabulaire spécifique sont bien reconnues** : électrocution (classe 2, $F1 = 0,88$) et chutes de hauteur (classe 5, $F1 = 0,85$).
- **Les classes rares sont mal reconnues** : la classe 4 ne compte que 2 incidents de test et n'est jamais prédite correctement, et la classe 1 n'en compte que 7. Avec si peu d'exemples, le modèle a peu d'occasions d'apprendre leur vocabulaire.
- **Les classes 0, 7 et 8 se confondent** : 17 incidents de la classe 7 et 12 incidents de la classe 8 sont prédits en classe 0 (Figure 1). Ces trois classes décrivent des blessures mécaniques et partagent une partie de leur vocabulaire (section 2.3).

Surapprentissage et régularisation : L'exactitude de 100 % à l'entraînement montre que la régression logistique mémorise les données. La Table 8 montre l'effet d'une régularisation plus forte, comparée par validation croisée sans utiliser le test.

C	Validation croisée	Entraînement
0,01	67,6 %	82,7 %
0,1	71,7 %	97,4 %
1 (défaut)	71,4 %	100,0 %
10	70,7 %	100,0 %

TABLE 8 – Effet de la régularisation de la Régression Logistique.

Avec $C = 0,1$, l'écart entre l'entraînement et la validation se réduit sans perte de performance, alors qu'une régularisation trop forte ($C = 0,01$) fait chuter les résultats. Le gain reste faible (0,3 point) : le surapprentissage observé pénalise peu la généralisation sur ce jeu de données.

2.3 Explicabilité et interprétation des classes

Pour interpréter les classes, nous avons extrait, pour chaque classe, les dix mots qui ont le plus de poids dans chaque modèle : `coef_` pour la régression logistique et `feature_log_prob_` pour le Naïf Bayésien.

```
Mots les plus importants par classe (Naïve Bayes) :

Classe 0 : the, was, and, employee, to, of, on, in, he, at
Classe 1 : the, employee, was, and, to, on, of, in, he, water
Classe 2 : the, was, and, employee, to, of, on, he, an, in
Classe 3 : the, and, to, employee, was, of, on, in, he, were
Classe 4 : the, and, was, to, employee, he, of, at, on, his
Classe 5 : the, was, employee, and, to, of, on, he, at, in
Classe 6 : the, and, was, employee, to, of, on, he, in, an
Classe 7 : the, was, employee, and, to, of, on, he, in, his
Classe 8 : the, was, and, to, employee, of, on, in, he, at
```

FIGURE 2 – Mots les plus importants par classe selon le Naïf Bayésien.

```
Mots les plus importants par classe (Régression Logistique) :

Classe 0 : collapsed, crushed, vehicle, trench, garage, run, crushing, ran, wall, loader
Classe 1 : drowned, barge, into, river, diver, coffer, dam, pool, boat, 12
Classe 2 : electrocuted, energized, contacted, power, volt, shock, contact, overhead, bus, electrical
Classe 3 : asphyxia, gas, asphyxiated, overcome, methylene, chloride, paint, 2011, inhalation, carbon
Classe 4 : heat, burned, hot, temperature, degrees, worker, fahrenheit, 2005, insulation, steam
Classe 5 : fell, balance, ladder, ft, fall, feet, scaffold, height, opening, fractured
Classe 6 : burns, fire, ignited, gas, explosion, exploded, tank, natural, fuel, hospitalized
Classe 7 : saw, finger, case, about, burns, hopper, serial, struck, directing, tools
Classe 8 : struck, striking, elevator, blocking, sustained, eye, beam, march, counterweight, load
```

FIGURE 3 – Mots les plus importants par classe selon la Régression Logistique.

Les deux modèles font ressortir des mots radicalement différents (Figures 2 et 3) :

- **Naïf Bayésien** : comme nous n'avons pas retiré les mots vides, les mots les plus fréquents de tout le corpus (*the, was, and*, mais aussi *employee*, présent dans presque toutes les descriptions) dominent toutes les classes. Ils ne permettent pas d'interpréter les étiquettes.
- **Régression Logistique** : un mot présent dans toutes les classes reçoit un poids proche de zéro, alors qu'un mot propre à une classe, comme *electrocuted* pour la classe 2, reçoit un poids élevé. Ses mots sont donc directement interprétables.

Nous avons donc interprété les classes à partir de la régression logistique, qui est aussi le modèle le plus performant (Table 9).

Classe	Interprétation	Mots discriminants (LR)
0	Éboulement / écrasement	<i>collapsed, crushed, trench, crushing, wall</i>
1	Noyade	<i>drowned, river, diver, pool, boat</i>
2	Électrocution	<i>electrocuted, energized, power, volt, shock</i>
3	Asphyxie / gaz	<i>asphyxia, gas, asphyxiated, inhalation, carbon</i>
4	Brûlures thermiques	<i>heat, burned, hot, temperature, steam</i>
5	Chutes de hauteur	<i>fell, balance, ladder, fall, scaffold</i>
6	Incendies / explosions	<i>burns, fire, ignited, explosion, exploded</i>
7	Blessures avec outils	<i>saw, finger, hopper, struck, tools</i>
8	Heurté par un objet / charge	<i>struck, elevator, beam, counterweight, load</i>

TABLE 9 – Interprétation des classes d'incidents à partir des mots les plus discriminants.

Classes difficiles à interpréter : Les classes 7 et 8 sont les moins nettes. Elles partagent le mot *struck*, et la classe 7 mêle des mots clairs (*saw, finger, tools*) à des mots sans lien évident

avec un type d'incident (*case*, *about*, *serial*). Notre interprétation repose donc sur une partie seulement de leurs mots. Ce recoupement est cohérent avec la matrice de confusion, où ces classes sont souvent confondues entre elles et avec la classe 0. La classe 1 contient elle aussi des mots peu informatifs (*into*, *12*), probablement parce qu'elle compte peu d'exemples d'entraînement.

Limites : Le sac de mots ignore l'ordre et le contexte des mots : « struck by a beam » et « struck with a tool » partagent le même mot-clé, alors qu'ils relèvent de classes différentes. Retirer les mots vides, pondérer les classes rares ou utiliser une représentation TF-IDF sont des pistes d'amélioration.

3 Tâche 3 – Classification d'incidents avec des Transformers

On reprend la classification des descriptions d'incidents avec deux modèles Transformer affinés grâce à HuggingFace : (`bert-base-uncased`), imposé, et ELECTRA (`electra-base-discriminator`).

3.1 Démarche

La méthodologie derrière le développement du modèle BERT suit les mêmes étapes décrites dans le cours des « Transformers -Hugging Face » : création du jeu de données, tokenisation, création du modèle, entraînement et évaluation.

- **Création du jeu de données :** après le chargement des trois partitions de données (données d'entraînement, de test et de validation), elles sont converties au format des données de HuggingFace contenant le texte de chaque incident et son étiquette (convertie en entier). Nous avons au total 9 classes.
- **Tokenisation :** dans cette étape nous utilisons le tokeniseur de BERT. BERT étant limité à 512 jetons, nous avons mesuré la longueur des textes d'entraînement : elle est en moyenne de 147 jetons (médiane : 119, 95^e centile : 342, maximum : 850). Seuls 1,2 % des textes dépassent 512 jetons ; nous fixons donc la longueur maximale à 512 et tronquons ces quelques textes.
- **Création du modèle :** nous partons du modèle BERT déjà préentraîné, auquel nous ajoutons une nouvelle tête de prédiction propre à notre tâche. À partir de la représentation du texte construite par BERT, cette tête donne un score à chacune des 9 classes d'incidents, et la classe ayant le score le plus élevé est retenue comme prédiction.
- **Entraînement :** nous entraînons ensuite le modèle sur les données d'entraînement, afin d'adapter BERT à notre tâche de classification. Le modèle parcourt plusieurs fois ces données (6 époques) et après chaque passage nous mesurons son exactitude (accuracy) sur les données de validation et conservons la version la plus performante. Les réglages utilisés (voir le code) suivent les valeurs habituellement recommandées pour BERT [1].
- **Évaluation :** le modèle BERT est évalué une seule fois sur la partition de test. La métrique utilisée est l'exactitude (accuracy), qui est la proportion d'incidents correctement classés.

Données : Le corpus comporte trois partitions : entraînement (2 475 incidents), validation (531) et test (531). La validation sert à choisir la meilleure époque ; le test n'est utilisé qu'une fois, à la fin.

Longueur des textes : BERT accepte au plus 512 jetons par texte. Nous avons donc d'abord mesuré la longueur des descriptions :

Moyenne	Médiane	95 ^e centile	Maximum	Textes > 512 jetons
147	119	342	850	1,2 %

TABLE 10 – Longueur des descriptions d’entraînement, en jetons BERT.

Comme seuls 1,2 % des textes dépassent la limite, nous gardons 512 jetons et tronquons ces quelques textes.

Choix du second modèle : L’énoncé exclut les variantes de BERT. Nous avons choisi ELECTRA, un encodeur anglais de même taille que BERT mais pré-entraîné différemment : au lieu de deviner des mots masqués, il apprend à repérer les mots remplacés dans un texte, ce qui lui permet d’apprendre de tous les mots.

Entraînement : Les deux modèles sont affinés avec exactement les mêmes hyperparamètres, pour que la comparaison soit équitable : 6 époques, taux d’apprentissage de 2×10^{-5} , lots de 8 exemples et graine fixée à 42. L’exactitude de validation est mesurée à chaque époque, et la meilleure époque est conservée à la fin.

3.2 Résultats

Époque	BERT		ELECTRA	
	Perte (val.)	Exactitude (val.)	Perte (val.)	Exactitude (val.)
1	0,803	73,4 %	0,844	73,8 %
2	0,628	80,8 %	0,722	80,4 %
3	0,597	82,5 %	0,706	81,7 %
4	0,678	81,7 %	0,832	81,4 %
5	0,745	82,9 %	0,872	81,7 %
6	0,773	82,5 %	0,893	81,7 %

TABLE 11 – Perte et exactitude sur la validation à chaque époque (en gras : minimum de perte et époque retenue).

Modèle	Exactitude (test)
Naïf Bayésien (tâche 2)	67,6 %
Régression Logistique (tâche 2)	69,3 %
BERT	77,0 %
ELECTRA	78,5 %

TABLE 12 – Exactitude de tous les modèles sur les mêmes données de test (531 incidents).

3.3 Analyse

Les Transformers dépassent nettement les modèles de la tâche 2 : BERT et ELECTRA gagnent respectivement 7,7 et 9,2 points d’exactitude par rapport à la régression logistique (Table 12). Contrairement au sac de mots, ils tiennent compte du contexte de chaque mot et profitent de leur pré-entraînement, ce qui les aide face aux formulations variées et aux classes peu représentées.

ELECTRA fait légèrement mieux que BERT : L'écart est de 1,5 point, soit environ 8 incidents sur 531. Il reste trop faible pour conclure avec certitude : un autre découpage ou une autre graine aléatoire pourrait le réduire.

Les deux modèles surapprennent après 3 époques : La perte de validation est minimale à l'époque 3, puis remonte (Table 11), alors que l'exactitude de validation reste stable autour de 82 %. Le modèle devient de plus en plus confiant, y compris dans ses erreurs. Grâce à la sélection de la meilleure époque, ce surapprentissage n'affecte pas le modèle final, mais trois ou quatre époques auraient suffi.

Écart entre validation et test : L'exactitude passe d'environ 82,9 % en validation à 77-78,5 % en test. Une partie de cet écart vient du choix de la meilleure époque sur la validation, ce qui rend ce score légèrement optimiste. Le test donne l'estimation la plus fiable.

Limites :

- **Le coût :** l'affinage prend de quelques minutes à plusieurs heures selon la machine, contre quelques secondes pour la régression logistique, pour un gain de 8 à 9 points.
- **La troncature :** 1,2 % des textes perdent leur fin.
- **L'interprétabilité :** contrairement à la régression logistique, un Transformer ne fournit pas directement de mots discriminants par classe.

Pistes d'amélioration : Ces résultats ouvrent plusieurs pistes, qu'il faudrait toutefois valider par des expériences supplémentaires :

- **Ajuster le nombre d'époques :** la perte de validation remontant dès l'époque 4, un entraînement plus court (3 ou 4 époques) ou un arrêt anticipé sur la perte de validation devrait limiter le surapprentissage, pour un coût moindre.
- **Contourner la limite de 512 jetons :** découper les descriptions les plus longues en plusieurs segments, puis combiner les prédictions, éviterait de perdre la fin de ces textes.
- **Optimiser les hyperparamètres de chaque modèle :** nous avons utilisé les mêmes réglages pour BERT et ELECTRA afin de les comparer équitablement, mais le taux d'apprentissage et la régularisation optimaux ne sont pas forcément les mêmes pour les deux. Ils devraient être choisis sur la validation, jamais sur le test.

Avant de conclure qu'une de ces pistes améliore réellement les résultats, il faudrait répéter chaque entraînement avec plusieurs graines aléatoires : l'écart actuel entre BERT et ELECTRA (1,5 point) montre que des différences de cet ordre peuvent relever du hasard.

Références

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics : human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.